This material was developed in the context of the Nordplus project NPHE-2025/10158, “Co-designing a Computational Thinking and Mathematics Curriculum Framework for Nordic-Baltic Preservice Teacher Education

Integrating Computational Thinking into Mathematics Education

Brief Introduction to the Package

Computational thinking Computational thinking (CT) has been incorporated into many European compulsory school curricula in recent years (see Bocconi et al., 2022, for an overview). However, integrating CT into mathematics teaching and learning has been challenging, and different country-specific approaches have emerged (Broley et al., 2024).

CT in mathematics education can be described as a structured problem-solving process that involves interrelated thinking processes, including algorithmic thinking, decomposition, pattern recognition, abstraction, and generalisation (Kallia et al., 2021). CT also involves transposition, which refers to formulating a mathematical solution in a way that enables it to be transferred to and carried out by another person or a machine.

This teaching package was developed for teacher educators (TEs) as a sequence of two classroom activities for preservice teachers (PSTs). The package was developed in response to a shared need across the Nordic-Baltic region regarding CT in PST education. Given the diversity of national curricula across the Nordic-Baltic region and beyond, we do not propose a curriculum-specific approach. Instead, based on a needs analysis and a review of the literature, we designed a flexible package consisting of two extensive classroom activities that teacher educators can adapt to their local contexts.

The activities intentionally provide different lenses on CT: an unplugged CT activity based on Bebras tasks and a programming-oriented activity using Scratch. In the latter, programming is considered a practice through which CT processes can be enacted and made visible.

The activities are structured using Van den Akker’s (2003) curricular spider web framework, which highlights the interrelated components of curriculum design. Accordingly, each activity

specifies its rationale, aims and learning objectives, content, learning activities, the TE’s role, assessment approach, grouping, time allocation, required prior knowledge, materials and resources, and possible follow-up activities. The activities are intended as starting points for engaging PSTs in CT and mathematics; teacher educators may extend, modify, or supplement them with additional activities according to local needs and goals.

The activities follow a broad problem-solving cycle in which PSTs explore a problem, develop or formulate a solution procedure, test and evaluate their solution, identify errors or limitations, and revise their approach. However, the activities do not prescribe a fixed sequence of CT components and processes. The components and processes that become relevant, such as decomposition, algorithmic thinking, pattern recognition, abstraction, generalisation, evaluation, and debugging, depend on the task, the solution strategy, and the classroom discussion.

Rather than functioning as a fixed lesson script, the package is intended as a flexible teaching resource. Teacher educators may adapt the activities according to PSTs’ prior knowledge, the mathematics curriculum, the available time and resources, and the age group for which the PSTs are preparing to teach. Some discussions or reflective tasks may also be extended or revisited in a follow-up lesson.

Table 1 provides an overview of the activities, including their mathematical and technical prerequisites, estimated duration, main CT processes, materials, and intended contribution to PST learning.

Table 1. *Overview of the two activities*

Feature	***Activity 1: Exploring CT through Bebras Cards***	***Activity 2: Exploring polygons through Scratch***
Main purpose / rationale	Develop PSTs’ initial understanding of CT and selected CT processes and their connections with mathematical reasoning.	Connect properties of regular polygons with programming through prediction, explanation, testing, debugging, and generalisation.
Prerequisites	Familiarity with the mathematics curriculum for the relevant grades; no technical prerequisites.	Basic knowledge of regular polygons and angles; basic familiarity with Scratch Motion and Control blocks and the Pen extension.
Duration	Approximately 90 minutes	3 × 45 minutes
Main CT processes	Algorithmic thinking/sequencing, pattern recognition, evaluation, and debugging	Decomposition, algorithmic thinking, pattern recognition, testing and debugging, abstraction, and generalisation
Materials needed	Selected Bebras cards, printed task sheets or slides, and exit	Internet-connected computers or tablets with Scratch, and a

	cards	shared display (projector)

The two activities provide complementary perspectives. The Bebras activity foregrounds computational reasoning without requiring programming and therefore demonstrates that CT should not be equated with coding or digital technology. PSTs must interpret information, identify patterns, formulate and evaluate procedures, and justify solutions. The Scratch activity, by contrast, enables mathematical ideas and algorithms to be expressed in an executable form. Through drawing regular polygons, PSTs can observe how repetition, variables, turning angles, and general rules are represented in code. Predictions can be tested immediately, while unexpected results provide opportunities for debugging and mathematical explanation. Together, the activities demonstrate that CT can be enacted through both unplugged problem solving and programming.

Teacher Educator’s (TE’s) role

TEs play an important role in helping PSTs understand the relationship between CT and mathematics. This intersection is not always easy to identify, because students often assume they already know what mathematics is and may therefore overlook computational ideas already embedded in familiar mathematical practices, such as decomposition, pattern recognition, abstraction, algorithmic reasoning, and debugging.

Across the package, the TE has to facilitate PSTs’ inquiry such that they can discern themselves rather than presenting CT processes as a list of definitions at the outset. The TE elicits PSTs’ strategies, selects examples for whole-class discussion, introduces relevant CT terminology when it becomes meaningful, and supports PSTs in connecting CT processes with mathematical reasoning and teaching. The TE may adapt the depth and timing of explicit CT discussions according to the PSTs’ prior knowledge and the available time. A further responsibility is to shift the focus from completing a task successfully to explaining how and why a solution works, how it can be tested or improved, and how the activity could be adapted for pupils in different grade levels.

Therefore, the pedagogical challenge is thus one of **discernment**. By directing attention to critical features of CT and providing a conceptual language for describing them. The TEs can help PSTs develop a more nuanced understanding of how CT can contribute to mathematical meaning-making rather than being reduced to programming or the use of digital tools.

Assessment

Assessment of CT can be used not only to judge whether students reach a correct answer, but also to make their thinking visible. For TEs, it can reveal how PSTs use CT when solving mathematical problems, for example through decomposition, pattern recognition, step-by-step procedures, testing, and revision. By asking PSTs to explain, compare, document, and reflect on their strategies, assessment provides insight into how they conceptualize both mathematics and CT.

Assessment also has a formative role. Reflection can help PSTs become more aware of their own reasoning and recognize how CT components and processes contribute to their problem solving process. This is particularly important in mathematics, where identical answers may result from very different assumptions. Assessment can therefore illuminate how computational and mathematical thinking interact and develop together.

References

Bocconi, S., Chioccariello, A., Kampylis, P., Dagienė, V., Wastiau, P., Engelhardt, K., Earp, J., Horvath, M. A., Jasutė, E., Malagoli, C., Masiulionytė-Dagienė, V., & Stupurienė, G. (2022). *Reviewing computational thinking in compulsory education: State of play and practices from computing education*. Publications Office of the European Union. https://doi.org/10.2760/126955

Broley, L., Buteau, C., Modeste, S., Rafalska, M., & Stephens, M. (2024). Computational thinking and mathematics. In B. Pepin, G. Gueudet, & J. Choppin (Eds.), *Handbook of digital resources in mathematics education*. Springer. https://doi.org/10.1007/978-3-031-45667-1_12

Kallia, M., van Borkulo, S. P., Drijvers, P. H. M., Barendsen, E., & Tolboom, J. (2021). Characterising computational thinking in mathematics education: A literature-informed Delphi study. *Research in Mathematics Education*, 23(2), 159–187. https://doi.org/10.1080/14794802.2020.1852104

van den Akker, J. (2003). Curriculum perspectives: An introduction. In J. van den Akker, W. Kuiper, & U. Hameyer (Eds.), *Curriculum landscapes and trends* (pp. 1–10). Kluwer Academic Publishers. https://doi.org/10.1007/978-94-017-1205-7_1

Further Reading

Benton, L., Hoyles, C., Kalas, I., & Noss, R. (2017). Bridging primary programming and mathematics: Some findings of design research in England. *Digital Experiences in Mathematics Education*, *3*(2), 115–138. https://doi.org/10.1007/s40751-017-0028-x

Brennan, K., & Resnick, M. (2012). New frameworks for studying and assessing the development of computational thinking. *Paper presented at the Annual Meeting of the American Educational Research Association*, Vancouver, Canada. https://scratched.gse.harvard.edu/ct/files/AERA2012.pdf

Broza, O., Biberman-Shalev, L., & Chamo, N. (2023). “Start from scratch”: Integrating computational thinking skills in teacher education program. *Thinking Skills and Creativity*, *48*, 101285. https://doi.org/10.1016/j.tsc.2023.101285

Rajapakse Mohottige, N. U. S., Bjerke, A. H., & Andersen, R. (2024). Teacher education as stakeholder: teacher educator perspectives on the integration of computational thinking into

mathematics and science courses. *Teachers and Teaching*, *30*(4), 437-451. https://doi.org/10.1080/13540602.2024.2313635

Sentance, S., Waite, J., & Kallia, M. (2019). Teaching computer programming with PRIMM: A sociocultural perspective. *Computer Science Education*, *29*(2–3), 136–176. https://doi.org/10.1080/08993408.2019.1608781

Wing, J. M. (2006). Computational thinking. *Communications of the ACM*, *49*(3), 33–35. https://doi.org/10.1145/1118178.1118215

Yadav, A., Stephenson, C., & Hong, H. (2017). Computational thinking for teacher education. *Communications of the ACM*, *60*(4), 55-62. https://doi.org/10.1145/2994591